

OmniUI Fabric

Platform Specification v1.0

Experience OS for AI Agents and Enterprise Applications

Neo4j Graph Brain

A2UI Output

AG-UI Transport

MCP Gateway

Deterministic Runtime

Multi-Tenant

<10ms

p95 Render Latency

0

Runtime Tokens

>95%

Cache Hit Rate

12

Node Types

15+

Edge Types

1 Vision and Problem Statement

Why OmniUI Fabric exists

The Problem: Fragmented Experiences

Organizations maintain separate user experience implementations for every channel. The same loan application form is built in React for web, Swift for iOS, Kotlin for Android, a conversational flow for chatbot, and a card format for copilot surfaces. Every change must propagate to all five implementations.

Channel	Current Implementation	Pain Point
Web Portal	React / Angular / Vue component	Framework-specific, hard to reuse
Mobile App	Native Swift / Kotlin / RN	Different validation rules
Chatbot	Text-only conversational flow	No rich UI, poor UX
Copilot / AI Agent	Custom card per agent host	Bespoke per platform
Dashboard	Separate analytics view	No single source of truth

The Solution: Experiences as Data

OmniUI Fabric treats experiences as structured data — not code. Define what an experience *is* (intent, data, actions, rules, governance) once. The platform renders it consistently everywhere with near-zero runtime AI cost.

★ Core thesis: AI is used exactly once — at authoring time — to help define the experience. Every subsequent render is deterministic, cached, and consumes zero LLM tokens. Same inputs always produce same outputs.

Design Principles

#	Principle	What it means
1	Experiences as data	XD's are manifests, not code
2	Deterministic runtime	Same inputs → same A2UI output, always
3	AI only at authoring time	Zero tokens during render
4	Skills as semantic layer	Agents think in skills, not screens
5	Graph as the brain	Neo4j stores all domain knowledge with org_id isolation
6	Protocol-native	Output A2UI format, deliver over AG-UI transport
7	Governance first	Compliance rules encoded, not suggested

2 Core Concepts

Skills · XDs · Runtime · MCP Gateway · Registries

Skills	Reusable, composable units of work. What can happen here? Channel-neutral, versioned, composable, governed, fallback-aware. Examples: apply-for-loan, verify-identity, check-application-status.
Experience Definitions (XDs)	The UI implementation of a skill. Specifies template, data slots, channel support, rendering rules, and actions. One skill → multiple XDs (one per channel, or one XD covering multiple channels).
Runtime Engine	A pure deterministic function: evaluate(skill, xd, context) → A2UI. No LLM inference. Evaluates rules, resolves bindings, assembles A2UI JSON. Fully cacheable. Target: <10ms p95 latency, 0 tokens.
MCP Gateway	Exposes OmniUI Fabric as MCP tools any AI agent can invoke: find_skill, execute_skill, dispatch_action, get_skill_graph, create_experience_definition, validate_experience_definition, subscribe_to_actions.
Registries (Neo4j)	All six registries are backed by Neo4j: Skill Registry, XD Registry, Template Library, Data Schema Registry, Agent Capability Graph, Governance Registry. Every node carries org_id for multi-tenant isolation.

3 Protocol Stack

OmniUI Fabric above A2UI above AG-UI

OmniUI Fabric sits above the emerging agentic UI protocol stack. It does not compete with AG-UI or A2UI — it uses them as its output format and transport layer respectively.

OmniUI Fabric

Skills · XD Registry · Deterministic Runtime · Rules Engine · Neo4j Graph Brain
"WHAT to render, for WHOM, governed HOW"

outputs A2UI component tree

A2UI (Google)

Declarative component format (JSON) · Cross-platform: Flutter · Angular · Lit · SwiftUI
"WHAT the UI looks like — the payload"

delivered over AG-UI events

AG-UI (CopilotKit / Microsoft)

Event streaming protocol (SSE · WebSocket) · Human-in-the-loop · State sync
"HOW the UI gets to the user — the transport"

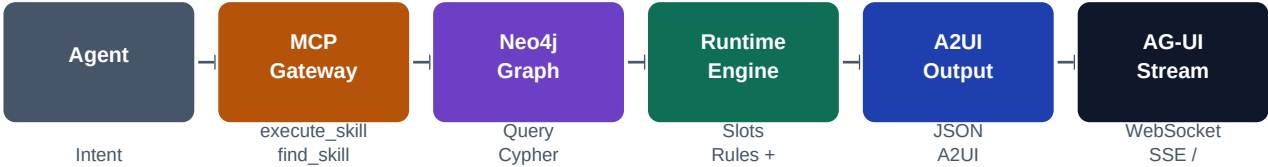
Protocol	Owner	Layer	What OmniUI Fabric uses it for
A2UI	Google	Payload format	Runtime outputs A2UI JSON component trees — declarative, not executable
AG-UI	CopilotKit / Microsoft	Transport	A2UI payloads delivered via AG-UI SSE/WebSocket event streams
MCP	Anthropic / Community	Agent interface	OmniUI Fabric exposes 7 MCP tools for agent orchestration

→ Key insight: A2UI uses a declarative, safe-like-data format where the client maintains a catalog of trusted components. OmniUI Fabric's XD slot types map directly to A2UI components, including custom org-specific components (OmniUI.CurrencyInput, OmniUI.RangeSlider) registered in the client catalog.

4 Architecture

System topology and request flow

Request Flow: Agent → MCP → Neo4j → Runtime → A2UI → AG-UI



End-to-End Request Flow

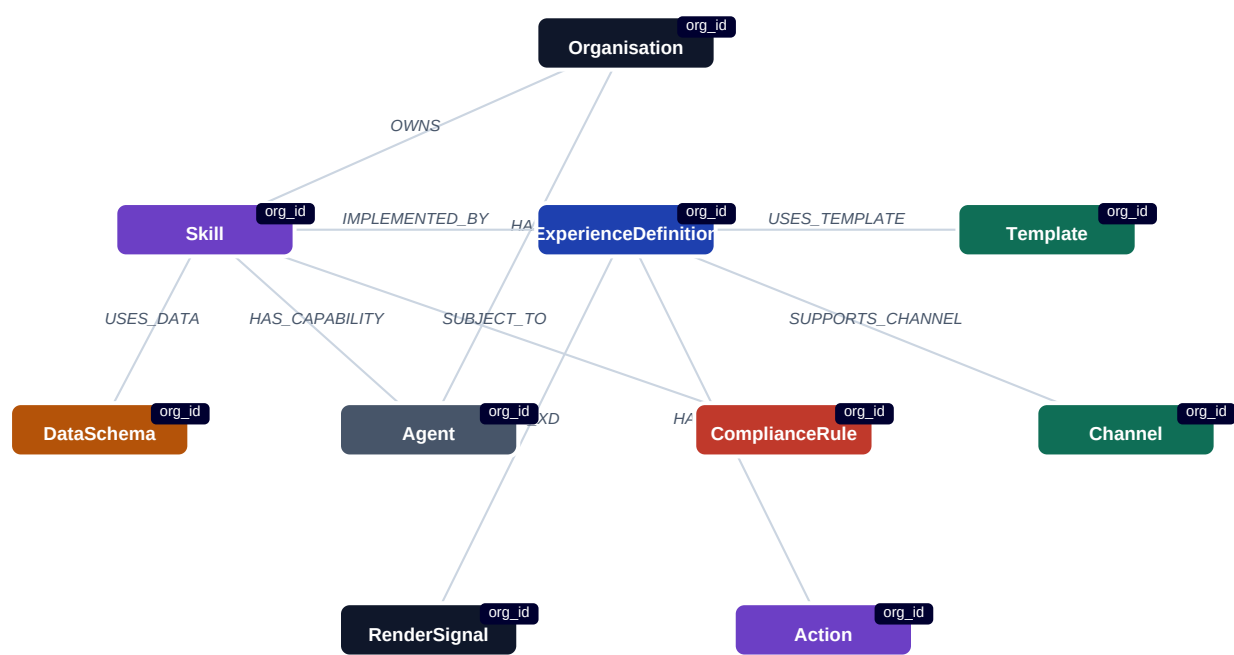
Step	Actor	Action	Output
1	Agent	Calls find_skill("apply-for-loan", channel="chatbot")	MCP tool invocation
2	MCP Gateway	Queries Neo4j with org_id filter	Skill node + XD bindings
3	Neo4j	Returns skill metadata, IMPLEMENTED_BY edges	Skill + XD manifest
4	Agent	Calls execute_skill with context object	Render request
5	Runtime	Checks Redis cache (key = hash of inputs)	Cache HIT → return immediately
6	Runtime	Evaluates CEL rules against context	Matched rules list
7	Runtime	Accumulates patches, resolves slots	Resolved render tree
8	Runtime	Serialises as A2UI JSON component tree	A2UI payload
9	AG-UI	Streams A2UI payload to frontend client	SSE/WebSocket event
10	Client	Renders A2UI using native framework components	Native UI (0 iframes)
11	User	Submits action → dispatch_action webhook	Agent receives result, transitions

5 Neo4j Graph Data Model

Nodes, attributes, edges — multi-tenant by org_id

- Multi-tenancy: Every node carries org_id: String. ALL Cypher queries MUST filter by org_id. No node is accessible across tenant boundaries without an explicit cross-org TRUSTS edge. This is enforced at the query layer, not the application layer.

5.1 Node Types



Neo4j node relationship overview (all nodes carry org_id)

NODE: Organisation	Type	Description
org_id	String	Unique primary key (e.g. acme-bank)
name	String	Display name
plan	Enum	free professional enterprise
status	Enum	active suspended archived
compliance_level	Enum	standard gdpr hipaa pci
created_at	DateTime	ISO 8601 timestamp
settings	JSON	Feature flags, org-level defaults

NODE: Skill	Type	Description
skill_id	String	Unique within org (e.g. apply-for-loan)
org_id	String	Tenant isolation key
name	String	Human-readable name
domain	String	financial-services hr ecommerce ...
version	String	Semver: 2.1.0
status	Enum	draft staging stable deprecated archived
input_schema	JSON	JSON Schema for skill inputs
output_schema	JSON	JSON Schema for skill outputs
compliance_tags	String[]	GDPR PII_handling HIPAA ...
ai_fallback_enabled	Boolean	Allow LLM fallback if no XD
estimated_tokens_ui	Integer	Always 0 for deterministic path
estimated_tokens_fallback	Integer	Token budget for AI fallback
content_hash	String	SHA-256 of canonical manifest JSON
is_composite	Boolean	Has child skills (multi-step workflow)

NODE: ExperienceDefinition	Type	Description
xd_id	String	Unique within org
org_id	String	Tenant isolation key
implements_skill	String	skill_id reference
template_id	String	template_id reference
version	String	Semver
status	Enum	draft staging stable deprecated archived
channels	String[]	web mobile chatbot copilot portal
slots	JSON	Array of slot definitions (type, bind, compute, rules)
rules	JSON	Array of {id, condition, priority, patch}
actions	JSON	Array of action definitions with payload schemas
a2ui_mapping	JSON	slot_id → A2UI component type mapping
content_hash	String	SHA-256 of canonical manifest

NODE: Template	Type	Description
template_id	String	Unique; system templates prefixed sys:
org_id	String	_system for built-in templates
domain	String	financial hr ecommerce generic
version	String	Semver
slot_types	String[]	Which slot types this template supports
channel_support	String[]	Certified channels
html_template	String	Handlebars/Mustache template string
a2ui_components	JSON	Slot type → A2UI component mapping

NODE: DataSchema	Type	Description
schema_id	String	Unique within org
org_id	String	Tenant isolation key
version	String	Semver
json_schema	JSON	Full JSON Schema definition
pii_fields	String[]	Field names containing PII data
sensitive_fields	String[]	Fields requiring extra protection
classification	Enum	public internal confidential restricted

NODE: Agent	Type	Description
agent_id	String	Unique within org
org_id	String	Tenant isolation key
agent_type	Enum	claude openai gemini custom copilot
version	String	Agent version
channels	String[]	Supported delivery channels
status	Enum	active inactive deprecated
compliance_certifications	String[]	GDPR SOC2 HIPAA ...
estimated_tokens_with_ui	Integer	Token cost with XD rendering
estimated_tokens_without_ui	Integer	Token cost text-only

NODE: ComplianceRule	Type	Description
rule_id	String	Unique within org
org_id	String	Tenant isolation key
regulation	String	GDPR HIPAA PCI-DSS SOC2 ...
enforcement	Enum	blocking warning audit_only automatic
condition	String	CEL expression evaluated at runtime
action	JSON	What to do when rule is triggered
applies_to	String[]	channels roles domains

NODE: Channel	Type	Description
channel_id	String	web mobile chatbot copilot portal dashboard
org_id	String	Tenant isolation key
type	Enum	web mobile chatbot copilot portal voice iot
constraints	JSON	max_width, max_payload_kb, supports_js, ...
a2ui_renderer	String	Which A2UI renderer this channel uses
agui_transport	Enum	sse websocket webhook polling

NODE: RenderSignal	Type	Description
signal_id	String	UUID
org_id	String	Tenant isolation key
skill_id	String	Executed skill
xd_id	String	Rendered XD
channel	String	Delivery channel
rules_applied	String[]	IDs of rules that fired
render_time_ms	Float	Wall-clock render time
tokens_used	Integer	0 for deterministic; >0 only for AI fallback
cache_hit	Boolean	Was result served from cache?
ai_fallback_used	Boolean	Did runtime invoke an LLM?

5.2 Edge Types

Edge Type	From	To	Key Properties
IMPLEMENTED_BY	Skill	ExperienceDefinition	channel: String, impl_type: String, is_primary: Boolean
CHILD_SKILL	Skill	Skill	order: Integer, required: Boolean
USES_DATA	Skill	DataSchema	mode: read write read_write, fields: String[], required: Boolean
SUBJECT_TO	Skill	ComplianceRule	inherited: Boolean
USES_TEMPLATE	ExperienceDefinition	Template	template_version: String, customizations: JSON
HAS_ACTION	ExperienceDefinition	Action	order: Integer, slot_id: String
SUPPORTS_CHANNEL	ExperienceDefinition	Channel	layout: String, max_width: Integer, compact_mode: Boolean
HAS_CAPABILITY	Agent	Skill	granted_at: DateTime, permission: execute read_only, expires
BELONGS_TO	Agent	Organisation	—
RESTRICTED_BY	DataSchema	ComplianceRule	field_names: String[], channel_constraint: String[]
MEMBER_OF	User	Team	role: String
OWNS	Team	Skill	domain: String
PUBLISHED	User	Skill/XD	at: DateTime
FOR_XD	RenderSignal	ExperienceDefinition	—
SUPERSEDES	SkillVersion	SkillVersion	—
TRUSTS	Organisation	Organisation	scope: String[], since: DateTime (cross-org, enterprise only)

5.3 Key Cypher Queries

Find stable skills for a channel

```
MATCH (s:Skill {org_id: $org_id, status: "stable"}) -[:IMPLEMENTED_BY {channel: $channel}]->
(xd:ExperienceDefinition {status: "stable"}) RETURN s, xd ORDER BY s.domain, s.name
```

Impact analysis — XD deprecation

```
MATCH (xd:ExperienceDefinition {xd_id: $xd_id, org_id: $org_id}) <-[:IMPLEMENTED_BY]-(s:Skill)
<-[:HAS_CAPABILITY]-(a:Agent) RETURN xd.xd_id, s.skill_id, collect(a.agent_id) AS affected_agents
```

Skills with PII subject to GDPR

```
MATCH (s:Skill {org_id: $org_id})-[:USES_DATA]->(ds:DataSchema)
-[:RESTRICTED_BY]->(cr:ComplianceRule {regulation: "GDPR"}) WHERE size(ds.pii_fields) > 0 RETURN
s.skill_id, ds.schema_id, ds.pii_fields, cr.enforcement
```

Full composite skill tree

```
MATCH path = (parent:Skill {skill_id: $id, org_id: $org_id}) -[:CHILD_SKILL*1..5]->(child:Skill)
RETURN path ORDER BY [r IN relationships(path) | r.order]
```

Dead rules (never fired in 30 days)

```
MATCH (xd:ExperienceDefinition {org_id: $org_id}) WHERE NOT EXISTS { MATCH
(sig:RenderSignal)-[:FOR_XD]->(xd) WHERE ANY(r IN sig.rules_applied WHERE r IN [rule IN xd.rules |
rule.id]) } RETURN xd.xd_id, "no rules fired in 30 days" AS warning
```

6

Functional Requirements

Skills · XDs · Runtime · MCP · Studio · Governance

6.1 Skill Management

ID	Requirement
SK-01	Author can create a skill via AI chat (natural language → manifest)
SK-02	Skill must have org_id, skill_id, version, status, input_schema, output_schema
SK-03	Skill versioning follows semver; published versions are immutable
SK-04	Skill can declare child skills and transition rules (composite workflows)
SK-05	Skill must declare at least one XD implementation or a text fallback
SK-06	Status transitions: draft → staging → stable → deprecated → archived
SK-07	Deprecated skill notifies all agents with HAS_CAPABILITY edge
SK-08	Skill search supports keyword, domain, category, and channel filters
SK-09	find_skill excludes skills with blocking compliance rules for the channel
SK-10	Content hash computed on canonical JSON form at publish time

6.2 Runtime Rendering

ID	Requirement
RT-01	Runtime is a pure function: same inputs always produce same A2UI output
RT-02	Runtime performs no LLM inference; zero tokens consumed
RT-03	p95 render latency < 10ms (cache miss); < 1ms (cache hit)
RT-04	Rules evaluated using CEL (Common Expression Language)
RT-05	Compute expressions use safe formula engine: PMT, ROUND, IF, MIN, MAX, CONCAT
RT-06	Data bindings use JSONPath against the context object
RT-07	Slots with visible_if=false are excluded from A2UI output entirely
RT-08	Cache key = SHA-256(xd_id + xd_version + org_id + context fields in rules)
RT-09	Cache TTL = 1 hour default; invalidated on XD publish
RT-10	Every render emits a RenderSignal node to Neo4j (async, non-blocking)
RT-11	AI fallback invoked only if ai_fallback_enabled:true AND no XD matches
RT-12	Output format is A2UI JSON component tree (v0.8+)

ID	Requirement
RT-13	A2UI output delivered over AG-UI SSE/WebSocket event stream

6.3 MCP Gateway

ID	Requirement
MCP-01	Gateway exposes 7 tools: find_skill, execute_skill, dispatch_action, get_skill_graph, create_experience_definition, validate_experience_definition, subscribe_to_actions
MCP-02	All tool calls require org_id in context or auth JWT token
MCP-03	find_skill returns ranked list with confidence scores
MCP-04	execute_skill returns A2UI JSON + AG-UI stream endpoint URL
MCP-05	dispatch_action validates payload against action payload_schema first
MCP-06	subscribe_to_actions returns session token + webhook registration URL
MCP-07	All MCP tool calls are logged as RenderSignal or audit nodes in Neo4j

6.4 Governance

ID	Requirement
GV-01	Compliance rules stored as ComplianceRule nodes in Neo4j
GV-02	Blocking rules prevent skill execution on restricted channels
GV-03	PII fields in DataSchema.pii_fields stripped from A2UI output when data-minimization rule applies
GV-04	Every rule evaluation logged in RenderSignal.rules_applied
GV-05	Promotion pipeline: draft → staging (automated CI) → production (human approval)
GV-06	Breaking version changes require explicit approval from org admin
GV-07	Deprecated assets trigger notification to all dependent Agent nodes

7 Non-Functional Requirements

Performance · Reliability · Security · Cost

<10ms

p95 Render Latency
(cache miss)

<1ms

p95 Render Latency
(cache hit)

0

Runtime Tokens
(deterministic path)

>95%

Cache Hit Rate
(stable XDs)

99.9%

API Uptime Target

Metric	Target	Notes
Render latency (cache hit)	< 1ms	Served from Redis edge cache
Render latency (cache miss p95)	< 10ms	Full CEL evaluation + A2UI serialization
Render latency (cache miss p99)	< 50ms	Complex rule sets
MCP tool call (end-to-end)	< 100ms	Including Neo4j query + render
Neo4j query (indexed read)	< 5ms	With org_id + status composite index
Throughput per node	> 10,000 renders/sec	Stateless runtime, horizontal scale
Cache hit rate (stable XDs)	> 95%	Context combination space is finite
AI fallback rate	< 5% of renders	Alert threshold for incomplete rule coverage
Authoring Studio first token	< 1s	Claude API streaming response

Security Requirements

- All API calls authenticated with JWT; org_id extracted from token claims
- Neo4j queries always include org_id filter — enforced at query layer, not application layer
- No cross-tenant data leakage is possible by design (query parameterization)
- A2UI output validated against component allowlist before delivery to client
- Webhook calls signed with HMAC-SHA256; recipient must verify before processing
- PII fields masked in logs and RenderSignal nodes
- TLS 1.3 for all network traffic
- A2UI is declarative data, not executable code — no code injection risk

Cost Model

Scenario	Token Cost	Frequency
Standard render (deterministic)	0 tokens	>95% of all renders
AI fallback render	~500 tokens (skill-declared)	<5% of renders

Scenario	Token Cost	Frequency
Authoring Studio: skill generation	~2,000 tokens	One-time per skill
Authoring Studio: disambiguation Q&A	~500 tokens	Per authoring session
Target AI cost per 1,000 renders	< 25 tokens average	At <5% fallback rate

8 Runtime Engine

Deterministic pipeline · Fallback hierarchy · A2UI output

Rendering Pipeline (9 Steps)

Step	Action	Mechanism	Output
1	Cache lookup	Redis GET with SHA-256 cache key	HIT → return A2UI (<1ms)
2	Load XD manifest	Neo4j MATCH with org_id filter	XD definition + rules
3	Build context	Merge incoming + user + org defaults	Flat context object
4	Evaluate rules	CEL evaluator for each rule condition	Matched rules list
5	Accumulate patches	Apply by priority order (0→5)	Merged slot config
6	Resolve slots	JSONPath bind + formula compute + format	Resolved slot values
7	Map to A2UI	a2ui_mapping in XD manifest	A2UI component tree
8	Cache result	Redis SET with TTL	Cached for next request
9	Emit signal	Async write to Neo4j RenderSignal	Audit trail + analytics

Fallback Hierarchy

Level	Name	Trigger	Token Cost
0	Cache hit	Cache key found in Redis	0
1	Deterministic render	Standard path: rules + slots + A2UI	0
2	Slot default	Missing data binding → use declared default	0
3	Template fallback	No channel-specific rules → base template	0
4	Text fallback	No XD for channel → text from skill metadata	0
5	AI fallback	ai_fallback_enabled:true AND no XD match	~500

9

Testing Strategy

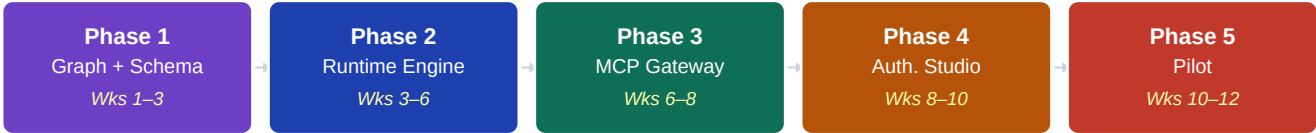
Unit · Integration · Contract · Performance · Security

Unit Tests	- Rules evaluator: assert correct match/no-match for 10+ context combinations per rule - Slot resolver: test each slot type's binding, compute, format, visible_if logic - Cache key generation: assert semantically equivalent contexts produce identical keys - A2UI serializer: assert output validates against A2UI v0.8 JSON Schema - CEL expressions: fuzz test with random context inputs; assert no panics or exceptions
Integration Tests	- Neo4j queries: seed test graph, assert each query returns expected subgraph - Runtime pipeline: end-to-end render test for each template × each channel - MCP tools: mock agent calls each tool; assert correct Neo4j state changes - Cache invalidation: publish new XD version; assert old cache entries are evicted - org_id isolation: assert query with org_id_A returns no nodes from org_id_B
Contract Tests	- A2UI conformance: every runtime output validated against A2UI v0.8 JSON Schema - AG-UI conformance: every event emitted validated against AG-UI event type schema - MCP conformance: all 7 tool schemas validated against MCP specification - Webhook payloads: assert HMAC-SHA256 signatures on all dispatch_action calls
Performance Tests	1,000 concurrent requests for same XD: assert p95 < 10ms 1,000 concurrent requests for 100 different XDs: assert p95 < 50ms - Neo4j under 10,000 skill nodes: assert indexed reads < 5ms - Cache hit rate > 95% over 10,000 identical context requests for stable XD - RenderSignal write: assert non-blocking (< 1ms added to render latency)
Security Tests	- Assert no Cypher query returns nodes from a different org_id - Assert PII fields stripped from A2UI output when compliance rule applies - Assert webhook signature validation rejects tampered payloads - Assert AI fallback cannot trigger without ai_fallback_enabled:true on Skill node - Assert blocking ComplianceRule prevents execute_skill from returning a bundle

10 MVP Scope and Roadmap

12-week build plan · Success metrics · In/Out scope

MVP Build Sequence — 12 Weeks



What's In MVP

Component	MVP Implementation
Neo4j Graph	All 12 node types, 15+ edge types, constraints, indexes from \$5
XD Schema	Slots, rules, channels, actions (no i18n, no state machine in v1)
Runtime	Full CEL rules engine, slot resolution, A2UI output, Redis cache
MCP Gateway	3 tools: find_skill, execute_skill, dispatch_action
A2UI Output	Full A2UI v0.8 component tree (system component catalog)
AG-UI Transport	SSE delivery of A2UI events
Authoring Studio	AI chat pane + YAML editor + live validation + preview button
Channels	Web + chatbot/copilot (2 channels)
Templates	5 canonical: single-step form, multi-step form, calculator, metric card, confirmation
Multi-tenancy	org_id on every node; JWT-based auth

Deferred to v2

Component	Why Deferred
Full Authoring Studio UI	Rules workbench, preview bench, publish console — need MVP validation first
Semantic vector search	find_skill uses keyword search in MVP; embeddings after schema stabilizes
Mobile and portal channels	Validate web + chatbot first
Fabric Mesh CDN	Single-region Redis in MVP; multi-region after pilot
Policy engine enforcement	Rules in graph but not enforced at runtime in MVP
subscribe_to_actions MCP tool	Webhook-based dispatch_action covers MVP needs
Cross-org skill sharing	TRUSTS edge modeled but not exposed in API

MVP Success Metrics



0

Appendices

Slot types · Rules · Compute · Protocols

A Slot Types (MVP)

Type	A2UI Component	Purpose
text-input	TextField	Free-form text entry
currency-input	OmniUI.CurrencyInput	Money with locale-aware formatting + validation
select	Select	Dropdown choice from options list
checkbox	Checkbox	Boolean toggle
range-slider	OmniUI.RangeSlider	Numeric range with min/max/step
date-input	DatePicker	Date selection with locale format
metric-display	OmniUI.MetricDisplay	Read-only computed value (result of formula)
text-display	Text	Read-only label or status message
button	Button	Action trigger (submit cancel help)
group	Container	Logical grouping of sub-slots

B Rule Priority Tiers

Priority	Tier Name	Example Conditions	Wins Over
5	Emergency override	Legal blocks, compliance emergency patches	Everything
4	Channel	channel == "chatbot"	3 and below
3	Organisation policy	Tenant brand rules, approved template overrides	2 and below
2	Role / segment	user.role == "advisor", user.tier == "premium"	1
1	Locale / device	locale == "en-GB", device.width < 500	Base only
0	Base	Template defaults (always present, always lowest)	Nothing

C Compute Expression Functions (Sandboxed)

Function	Signature	Description
PMT	PMT(rate, nper, pv)	Loan payment calculation (Excel-compatible)
ROUND	ROUND(value, decimals)	Round to N decimal places

Function	Signature	Description
IF	IF(condition, true_val, false_val)	Conditional value selection
MIN	MIN(a, b)	Minimum of two values
MAX	MAX(a, b)	Maximum of two values
CONCAT	CONCAT(a, b, ...)	String concatenation
FORMAT	FORMAT(value, pattern)	Format number or date as string
LOOKUP	LOOKUP(table_id, key)	Look up value in a registered lookup table

D Protocol and Tool References

Protocol / Tool	Reference URL	Role in OmniUI Fabric
A2UI	a2ui.org · github.com/google/A2UI	Output format for Runtime Engine
AG-UI	ag-ui.com	Transport for A2UI payloads to frontend
MCP	modelcontextprotocol.io	Agent interface (7 exposed tools)
CEL	cel.dev	Expression language for rules engine
Neo4j Cypher	neo4j.com/docs/cypher-manual	Graph query language for all registries
JSON Schema	json-schema.org	Validation for XD manifests and skill schemas
Claude API	docs.anthropic.com	LLM for Authoring Studio AI Designer

→ This document is the authoritative specification for OmniUI Fabric v1.0. Send to /goal to bootstrap the full project. The Neo4j graph database is the central brain — all domain knowledge is stored as nodes and edges, segregated by org_id across the entire graph. Start with Phase 1: set up Neo4j with the constraints and indexes defined in §5.4, then seed with the 5 system templates.